



INSTITUTO POLITÉCNICO NACIONAL.

LICENCIATURA EN CIENCIAS DE LA INFORMÁTICA.

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y  
CIENCIAS SOCIALES Y ADMINISTRATIVAS.

REPORTE DE COGIDOS

ELABORADO POR:

JUÁREZ DE JESUS JULIO CESAR

HERNÁNDEZ ROMERO ALEXIS JAIN

MARROQUÍN TÉLLEZ ANTONIO

GRUPO: 3CM32

NOMBRE DEL DOCENTE:

YVENTZ ENTZANA GARDUÑO

ABSTRACCIÓN Y USO DE DATOS

FECHA: 2 DE MAYO DEL 2026

Índice.

Código 10 ----- Página 3

Código 13 ----- Página 5

Código 16 ----- Página 7

Código 17 ----- Página 9

Código 18 ----- Página 11

Código 19 ----- Página 13

Código 20 ----- Página 15

Código 21 ----- Página 17

Código 1 ----- Página 19

Código 2----- Página 21

Código 3 ----- Página 23

Código 4 ----- Página 25

Código 5 ----- Página 27

Código 6 ----- Página 29

Código 7 ----- Página 31

Código 7 BIS ----- Página 33

Código 8 ----- Página 35

Código 9 ----- Página 37

Código 10 ----- Página 39

## Código 10.

## Código Correcto

```
#include <iostream>
using namespace std;
int main(){
    int n[5];
    int *p=n;
    int suma=0;
    for(int i=0;i<5;i++){
        cin>>*(p+i);
    }
    for(int i=0;i<5;i++){
        suma+=*(p+i);
    }
    cout<<"Promedio
"<<suma/5.0<<endl;
}
```

## Código 10.

## Código Erróneo

```
#include <iostream>
using namespace std;
int main(){
    int n[5];
    int *p;
    int suma;
    for(int i = 0; i <= 5; i++){
        cin >> *(p + i);
    }

    for(int i = 0; i < 5; i++){
        suma += *(p + i);
    }
```

Código 10: Un posible error es que no se valida la entrada del usuario, lo que puede provocar fallos si se ingresan datos no numéricos. También hay duplicación del mismo código, lo cual es una mala práctica. Como solución, se puede validar la entrada con condicionales y eliminar redundancias. Además, al no estar orientado a objetos, se podría mejorar creando una clase que contenga el método de suma y manejar la operación desde un objeto.

## Código 13.

## Código Correcto

```
#include <iostream>

using namespace std;

struct Persona {

    string nombre, ap, am, genero;

    int edad;

};

int main() {

    Persona personas[3];

    for (int i = 0; i < 3; i++) {

        cout << "Nombre: ";

        cin >> personas[i].nombre;

        cout << "Apellido paterno: ";

        cin >> personas[i].ap;

        cout << "Apellido materno: ";

        cin >> personas[i].am;

        cout << "Genero: ";

        cin >> personas[i].genero;

        cout << "Edad: ";

        cin >> personas[i].edad;

        cout << endl;

    }

    cout << "\nLista de personas\n";

    for (const auto &p : personas) {

        cout << p.nombre << " "

            << p.ap << " "

            << p.am << " "

            << p.edad << " años\n";

    }

}
```

Código 13.

```
Código Erróneo

#include <iostream>

using namespace std;

struct Persona{

    string nombre;

    int edad;

};

int main(){

    Persona personas[2];

    Persona *p;

    for(int i=0;i<=2;i++){

        cin>>(p+i)->nombre;

        cin>>(p+i)->edad;

    }
```

Código 13: Puede ocurrir un error al realizar la división si el usuario ingresa cero como divisor. También no hay control de tipos ni validación de datos. La solución es agregar una condición que evite dividir entre cero y validar los datos ingresados. Además, debía ser programado a objetos, por lo que se podría implementar una clase calculadora que contenga todos los métodos.

## Código 16.

## Código Erróneo

```
#include <iostream>

using namespace std;

class Datos {
private:
    int arr[5], suma = 0, max, min;

    float promedio;
public:
    void ejecutar() {
        cout << "Ingresa 5 numeros:\n";

        for (int i = 0; i < 5; i++) {
            cin >> arr[i];

            suma += arr[i];

            if (i == 0) max = min = arr[i];

            else {
                if (arr[i] > max) max = arr[i];

                if (arr[i] < min) min = arr[i];
            }
        }

        promedio = suma / 5.0;

        cout << "\nSuma: " << suma
        << "\nPromedio: " << promedio
        << "\nMax: " << max
        << "\nMin: " << min << endl;
    }
};

int main() {
    Datos d;

    d.ejecutar();
}
```



## Código 16.

## Código Erróneo

```
#include <iostream>

using namespace std;

class IOperacion {
public:

    virtual void calcular() = 0;

    virtual void mostrar() = 0;

};

class Datos : public IOperacion {
private:

    int arr[5];

    int suma;

    float promedio;
public:

    void ingresar(){

        for(int i = 0; i <= 5; i++)

            cin >> arr[i];

    }

    void calcular(){

        for(int i = 0; i < 5; i++)

            suma += arr[i];

        promedio = suma / 5;

    }

}
```

Código 16: No presenta errores de ejecución, pero contiene detalles que afectan su precisión y claridad. Utiliza índices fuera del rango permitido y variables sin inicializar, lo que puede provocar resultados incorrectos o comportamiento indefinido. Además, la división entera afecta el cálculo del promedio. La solución es corregir los límites de los ciclos, inicializar correctamente las variables y usar tipos adecuados para obtener resultados precisos. También se podría mejorar la estructura aplicando encapsulación para hacer el código más claro y mantenible.

## Código 17.

## Código Correcto

```
#include <iostream>

using namespace std;

void merge(int arr[], int l, int m, int r) {

    int temp[100], i = l, j = m + 1, k = 0;

    while (i <= m && j <= r)

        temp[k++] = (arr[i] < arr[j]) ? arr[i++] : arr[j++];

    while (i <= m) temp[k++] = arr[i++];

    while (j <= r) temp[k++] = arr[j++];

    for (i = l, k = 0; i <= r; i++, k++)

        arr[i] = temp[k];

}

void mergeSort(int arr[], int l, int r) {

    if (l < r) {

        int m = (l + r) / 2;

        mergeSort(arr, l, m);

        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);

    }

}

void mostrar(int arr[], int n) {

    for (int i = 0; i < n; i++)

        cout << arr[i] << " ";

    cout << endl;

}

int main() {

    int arr[] = {7, 3, 1, 9, 2};

    mergeSort(arr, 0, 4);

    mostrar(arr, 5);

    return 0;

}
```

## Código 17.

## Código Erróneo

```
#include <iostream>
using namespace std;
void m(int a[], int l, int m, int r){
    int t[10], i=l, j=m+1, k=0;
    while(i<=m && j<=r) t[k++] = (a[i]<a[j])?a[i++]:a[j++];
    while(i<=m) t[k++]=a[i++];
    while(j<=r) t[k++]=a[j++];
    for(i=l,k=0;i<=r;i++,k++) a[i]=t[k];
void s(int a[], int l, int r){
    if(l<r){
        int m=(l+r)/2;
        s(a,l,m); s(a,m+1,r);
        m(a,l,m,r);
    }
}
```

Código 17: No presenta errores de ejecución, pero tiene algunos aspectos que pueden mejorarse. Utiliza un arreglo temporal de tamaño fijo, lo cual limita su escalabilidad si se trabajan con más datos. Además, los valores del arreglo están definidos de manera estática, reduciendo la interacción con el usuario. Aunque implementa orientación a objetos mediante una interfaz, el manejo del arreglo fuera de la clase rompe parcialmente la encapsulación. La solución es permitir el ingreso dinámico de datos, utilizar estructuras más flexibles para el almacenamiento temporal y encapsular mejor los datos dentro de la clase para lograr un diseño más limpio y mantenible.

Código 18.

Código Correcto

```
#include <iostream>

using namespace std;

int particion(int arr[], int low, int high) {

    int pivot = arr[high], i = low - 1;

    for (int j = low; j < high; j++) {

        if (arr[j] < pivot)

            swap(arr[++i], arr[j]);

    }

    swap(arr[i + 1], arr[high]);

    return i + 1;

}

void quickSort(int arr[], int low, int high) {

    if (low < high) {

        int pi = particion(arr, low, high);

        quickSort(arr, low, pi - 1);

        quickSort(arr, pi + 1, high);

    }

}
```

## Código 18.

## Código Erróneo

```
#include <iostream>

using namespace std;

void q(int a[], int l, int h){

    int p=a[h], i=l-1;

    for(int j=l;j<=h;j++) // ERROR

        if(a[j]>p) swap(a[++i],a[j]); // ERROR

    swap(a[i],a[h]); // ERROR

    if(l<h){

        q(a,l,i-1);

        q(a,i+1,h);

    }

}

int main(){

    int a[]={8,4,2,6,1};

    q(a,0,5); // ERROR

    for(int i=0;i<=5;i++) cout<<a[i]<<" "; // ERROR

}
```

Código 18: No presenta errores de ejecución, pero tiene limitaciones en flexibilidad. Los datos están definidos de forma estática y el arreglo se maneja fuera de la clase, lo que reduce la encapsulación. Además, la interfaz es incompleta al no incluir un método para mostrar resultados. La solución es permitir entrada dinámica, agregar métodos a la interfaz y mejorar la encapsulación..

## Código 19.

Código Correcto

```
#include <iostream>

using namespace std;

int main() {

    int arr[] = {5, 2, 9, 1};

    int idx[4];

    // Inicializar índices
    for (int i = 0; i < 4; i++)

        idx[i] = i;

    // Burbuja indirecto
    for (int i = 0; i < 3; i++)

        for (int j = 0; j < 3 - i; j++)

            if (arr[idx[j]] > arr[idx[j + 1]])

                swap(idx[j], idx[j + 1]);

    // Mostrar ordenado
    for (int i = 0; i < 4; i++)

        cout << arr[idx[i]] << " ";

}
```

Código 19.

```
Código Erróneo

#include <iostream>

using namespace std;

void b(int a[], int i[], int n){

    for(int x=0;x<=n;x++) i[x]=x; // ERROR

    for(int x=0;x<n;x++)

        for(int y=0;y<n-x;y++) // ERROR

            if(a[i[y]] < a[i[y+1]]) // ERROR

                swap(i[y],i[y+1]);

}

int main(){

    int a[]={5,2,9,1}, i[4];

    b(a,i,4);

    for(int x=0;x<=4;x++) // ERROR

        cout<<a[i[x]]<<" ";

}
```

Código 19: No presenta errores de ejecución, pero tiene algunas limitaciones. Los datos están definidos de forma estática y el arreglo se maneja fuera de la clase, lo que reduce la encapsulación. Además, depende de arreglos auxiliares externos, lo que puede dificultar su uso. La solución es permitir entrada dinámica de datos y encapsular mejor tanto los datos como los índices dentro de la clase.

## Código 20.

## Código Correcto

```
#include <stdio.h>

void merge(int a[], int idx[], int l, int m, int r) {
    int temp[100], i = l, j = m + 1, k = 0;
    while (i <= m && j <= r)
        temp[k++] = (a[idx[i]] < a[idx[j]]) ? idx[i++] : idx[j++];
    while (i <= m) temp[k++] = idx[i++];
    while (j <= r) temp[k++] = idx[j++];
    for (i = l, k = 0; i <= r; i++, k++)
        idx[i] = temp[k];
}

void mergeSort(int a[], int idx[], int l, int r) {
    if (l == 0) // inicializar índices una sola vez
        for (int i = 0; i <= r; i++)
            idx[i] = i;
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(a, idx, l, m);
        mergeSort(a, idx, m + 1, r);
        merge(a, idx, l, m, r);
    }
}

void mostrar(int a[], int idx[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[idx[i]]);
}
```



## Código 20.

## Código Erróneo

```
#include <stdio.h>

void s(int*a,int*i,int l,int r){
    if(l==0)for(int x=0;x<=r;x++)i[x]=x; // ERROR
    if(l<r){
        int m=(l+r)/2,x=l,y=m+1,k=0,t[10];
        while(x<m&& y<=r)
            t[k++]=(a[i[x]]>a[i[y]])?i[x++]:i[y++]; // ERROR
        while(x<=m)t[k++]=i[x++];
        while(y<=r)t[k++]=i[y++];
        for(x=l;x<=r;x++) i[x]=t[x]; // ERROR
        s(a,i,l,m); s(a,i,m+1,r);
    }
}
```

Código 20: No presenta errores de ejecución, pero tiene algunas limitaciones en diseño. Utiliza arreglos de tamaño fijo, lo que reduce su flexibilidad al trabajar con más datos. Además, el arreglo y los índices se manejan externamente, lo que afecta la encapsulación. También depende de condiciones internas para inicializar los índices, lo que puede generar confusión. La solución es permitir manejo dinámico de datos y encapsular mejor la lógica dentro de la estructura para un diseño más claro y mantenible.

## Código 21.

## Código Correcto

```
#include <stdio.h>

int p(int a[], int i[], int l, int h){
    int pv = a[i[h]], j = l - 1;

    for(int k = l; k < h; k++){
        if(a[i[k]] < pv){
            int t = i[++j]; i[j] = i[k]; i[k] = t;
        }
    }
    int t = i[j+1]; i[j+1] = i[h]; i[h] = t;
    return j + 1;
}

void q(int a[], int i[], int l, int h){
    if(l == 0) for(int k = 0; k <= h; k++) i[k] = k;
    if(l < h){
        int m = p(a,i,l,h);
        q(a,i,l,m-1);
        q(a,i,m+1,h);
    }
}

void m(int a[], int i[], int n){
    for(int k = 0; k < n; k++) printf("%d ", a[i[k]]);
}
```

## Código 21.

## Código Erróneo

```

#include <stdio.h>

int p(int*a,int*i,int l,int h){

    int piv=a[i[h]],x=l-1;

    for(int j=l;j<=h;j++) // ERROR

        if(a[i[j]]>piv){x++;int t=i[x];i[x]=i[j];i[j]=t;} // ERROR

    int t=i[x];i[x]=i[h];i[h]=t; // ERROR

    return x;

}

void q(void*arr,int*i,int l,int h){

    int*a=(int*)arr;

    if(l==0)for(int x=0;x<=h+1;x++)i[x]=x; // ERROR

    if(l<h){

        int pi=p(a,i,l,h);

        q(a,i,l,pi-1);

        q(a,i,pi+1,h);

    }

}

int main(){

    int a[]={6,3,9,1},i[4];

    q(a,i,0,4); // ERROR

    for(int x=0;x<=4;x++) printf("%d ",a[i[x]]); // ERROR

}

```

Código 21: No presenta errores de ejecución aparentes en algunos casos, pero contiene varios problemas lógicos y de límites que pueden provocar comportamiento indefinido. Se accede a posiciones fuera del rango del arreglo en los ciclos, lo que puede generar fallos. Además, la inicialización de índices está mal condicionada, lo que puede provocar reordenamientos incorrectos. También hay inconsistencias en los límites usados al llamar la función principal y al imprimir los resultados. La solución es corregir los rangos de iteración, asegurar la correcta inicialización de los índices y validar adecuadamente los límites del arreglo para evitar accesos inválidos.

## Código 1.

Código Correcto

```
#include <stdio.h>

typedef struct {
    int valor;
} Objeto;

void mostrar(Objeto *o){
    printf("Valor: %d\n", o->valor);
}

int main(){
    Objeto obj = {100};
    mostrar(&obj);
}
```

## Código 1.

```
Código Erróneo

#include <stdio.h>

typedef struct{void(*m)(void*);}I;
typedef struct{I*o;}B;
typedef struct{B b;int v;}O;

void m(void*o){
    O*p=o;
    printf("%d\n",p->v+1); // ERROR
}

int main(){
    I i={m};
    O o;
    o.b.o=&i; // ERROR
    o.v=100;
    o.b.o->m(o); // ERROR
    return 0;
}
```

Código 1: No presenta errores de compilación, pero tiene problemas de diseño y uso de punteros. La asignación de la interfaz no es del todo coherente, lo que rompe la abstracción. Además, el acceso al método puede generar confusión si no se inicializa correctamente. La solución es mantener una estructura clara en la asignación de la interfaz y asegurar el uso correcto de los punteros para evitar errores lógicos.

## Código 2.

Código Correcto

```
#include <stdio.h>

#define MAX 10

typedef struct {
    int data[MAX], size;
} ADT;

void insertar(ADT *a, int v){
    if(a->size < MAX)
        a->data[a->size++] = v;
    else
        printf("Error: estructura llena\n");
}

void mostrar(ADT *a){
    if(a->size == 0) return (void)printf("Estructura vacia\n");

    for(int i = 0; i < a->size; i++)
        printf("%d ", a->data[i]);
    printf("\n");
}
```

## Código 2.

```

Código Erróneo

#include <stdio.h>

#define M 10

typedef struct{void(*i)(void*,int);void(*m)(void*);}I;
typedef struct{I*o;}B;
typedef struct{B b;int d[M];int s;}A;

void i(void*o,int v){
    A*p=o;

    if(p->s<=M) p->d[p->s++]=v; // ERROR
}

void m(void*o){
    A*p=o;

    for(int x=0;x<=p->s;x++) // ERROR
        printf("%d ",p->d[x]);
}

int main(){
    I i={i,m};

    A a;

    a.b.o=&i; // ERROR

    a.s=1; // ERROR

```

Código 2: No presenta errores de compilación, pero contiene problemas de lógica y manejo de límites que pueden causar comportamiento incorrecto. La condición de inserción permite exceder el tamaño del arreglo, lo que puede generar desbordamiento. Además, el recorrido para mostrar los elementos accede a una posición fuera del rango válido. También la inicialización del tamaño no es adecuada, lo que afecta el funcionamiento de la estructura. La solución es corregir los límites de control, inicializar correctamente el tamaño y validar el acceso al arreglo para evitar errores de memoria.

## Código 3.

## Código Correcto

```
#include <stdio.h>

#define MAX 5

typedef struct {
    int arr[MAX], top;
} Pila;

void push(Pila *p, int v){
    if(p->top == MAX - 1) return (void)printf("Pila llena\n");
    p->arr[++p->top] = v;
}

int pop(Pila *p){
    if(p->top == -1) return printf("Pila vacia\n"), -1;
    return p->arr[p->top--];
}

void mostrar(Pila *p){
    if(p->top == -1) return (void)printf("Pila vacia\n");
    for(int i = p->top; i >= 0; i--){
        printf("%d ", p->arr[i]);
        printf("\n");
    }
}

int main(){
    Pila p = {.top = -1};

    push(&p, 1);
    push(&p, 2);
    push(&p, 3);
    mostrar(&p);

    printf("Pop: %d\n", pop(&p));
    mostrar(&p);
}
```



## Código 3.

## Código Erróneo

```
#include <stdio.h>

#define M 5

typedef
struct{void(*p)(void*,int);int(*o)(void*);void(*m)(void*);}I;
typedef struct{I*op;}B;
typedef struct{B b;int a[M];int t;}P;

void p(void*x,int v){
    P*y=x;
    y->a[++y->t]=v; // ERROR (sin
validar)
}

int o(void*x){
    P*y=x;
    return y->a[y->t--];
}

void m(void*x){
    P*y=x;
    for(int i=0;i<=y->t;i++) // ERROR
(ordén incorrecto)
        printf("%d ",y->a[i]);
}

int main(){
    I i={p,o,m};
    P p;
    p.b.op=&i; // ERROR
```

Código 3: No presenta errores de compilación, pero tiene problemas de lógica y manejo de memoria que pueden causar resultados incorrectos. No se valida el límite del arreglo al insertar, lo que puede provocar desbordamiento. Además, la condición de recorrido en la impresión no respeta correctamente el estado de la estructura. También la inicialización del tope no es adecuada para una pila, lo que afecta el comportamiento de las operaciones. La solución es corregir los límites, inicializar correctamente el tope y validar cada operación para evitar accesos inválidos.

Código 4.

Código Correcto

```

#include <stdio.h>

#define MAX 5

typedef struct {
    int arr[MAX], f, r;
} Cola;

void enqueue(Cola *c, int v){
    if(c->r == MAX - 1) return (void)printf("Cola llena\n");
    c->arr[++c->r] = v;
}

int dequeue(Cola *c){
    if(c->f > c->r) return printf("Cola vacia\n"), -1;
    return c->arr[c->f++];
}

void mostrar(Cola *c){
    if(c->f > c->r) return (void)printf("Cola vacia\n");
    for(int i = c->f; i <= c->r; i++)
        printf("%d ", c->arr[i]);
    printf("\n");
}

int main(){
    Cola c = {.f = 0, .r = -1};
    enqueue(&c, 10);
    enqueue(&c, 20);
    enqueue(&c, 30);

    mostrar(&c);
    printf("Se elimino: %d\n", dequeue(&c));
    mostrar(&c);

```

## Código 4.

## Código Erróneo

```
#include <iostream>

Using namespace std;

Int factorial(int n){
    If(n==0)
        Return 1;
    Return n * factorial(n-1); }

Int fibonacci(int n){
    If(n<=1)
        Return n;
    Return fibonacci(n-1) + fibonacci(n-2); }

Int multiplicacion(int a,int b){
    If(b==0)
        Return 0;
    Return a + multiplicacion(a,b-1); }

Int potencia(int a,int b){
    If(b==0)
        Return 1;
    Return a * potencia(a,b-1); }

Int division(int a,int b){
    If(a<b)
        Return 0;
    Return 1 + division(a-b,b); }

Int main(){
    Cout<<"Factorial de 5:
    "<<factorial(5)<<endl;

    Cout<<"Fibonacci de 6:
    "<<fibonacci(6)<<endl;
```

Código 4: Puede haber errores al capturar cadenas con cin, ya que no permite espacios. También el tamaño de los arreglos es fijo. La solución es usar getline y permitir tamaños dinámicos. Aunque usa struct, debía ser orientado a objetos, por lo que sería mejor usar clases con encapsulamiento.

## Código 5.

Codigo Correcto

```
#include <stdio.h>

#define MAX 5

// ===== INTERFAZ =====

typedef struct {
    void (*ingresar)(void*);
    void (*mostrar)(void*);
} ILista;

// ===== ABSTRACTA =====

typedef struct {
    ILista* ops;
} Base;

// ===== CLASE =====

typedef struct {
    Base base;
    int arr[MAX];
    int size;
} Lista;

// ===== IMPLEMENTACION =====

// INGRESAR datos

void ingresar(void* obj){
    Lista* l = (Lista*)obj;
```

```
printf("¿Cuántos numeros
deseas ingresar? (max %d): ",
MAX);

    scanf("%d", &l->size);

    if(l->size > MAX){
        printf("Excede el limite. Se
ajustara a %d\n", MAX);

        l->size = MAX;
    }

    for(int i = 0; i < l->size; i++){
        printf("Numero %d: ", i + 1);
        scanf("%d", &l->arr[i]);
    }
}

// MOSTRAR datos

void mostrar(void* obj){
    Lista* l = (Lista*)obj;

    if(l->size == 0){
        printf("Lista vacia\n");

        return;
    }
}
```

Código 5.

```

Código Erróneo

#include <stdio.h>

#define MAX 5

// ===== INTERFAZ =====

typedef struct {

    void (*ingresar)(void*);

    void (*mostrar)(void*);

} ILista;

// ===== ABSTRACTA =====

typedef struct {

    ILista* ops;

} Base;

// ===== CLASE =====

typedef struct {

    Base base;

    int arr[MAX];

    int size;

} Lista;

// ===== IMPLEMENTACION =====

// INGRESAR datos

void ingresar(void* obj){

```

Código 5: Este programa muestra cómo usar una lista en un caso real:

- Permite registrar y gestionar elementos de una colección
- Aplica operaciones básicas como inserción y visualización de datos
  - Integra estructuras de datos con un enfoque modular

Es sencillo y funcional

## Código 6-

```
Código Correcto

: #include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*push)(void*, int);

    int (*pop)(void*);

    void (*mostrar)(void*);

} IPila;

// ===== ABSTRACTA =====

typedef struct {

    IPila* ops;

} Base;

// ===== NODO =====

typedef struct Nodo{

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo* top;

} Pila;
```

Código 6.

```
Código Erróneo

#include <stdio.h>
#include <stdlib.h>
// ===== INTERFAZ =====
typedef struct {
    void (*push)(void*, int);
    int (*pop)(void*);
    void (*mostrar)(void*);
} IPila;
// ===== ABSTRACTA =====
typedef struct {
    IPila* ops;
} Base;
// ===== NODO =====
typedef struct Nodo{
    int dato;
    struct Nodo* sig;
} Nodo;
// ===== CLASE =====
typedef struct {
    Base base;
    Nodo* top;
} Pila;
```

Código 6: Este programa muestra cómo usar una pila en un caso real:

- Permite almacenar datos dinámicamente mediante nodos enlazados
  - Aplica operaciones fundamentales como push y pop
- Integra estructuras de datos con un enfoque orientado a objetos

Es eficiente y práctico

Código 7.

Código Correcto

```
#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*enqueue)(void*, int);

    int (*dequeue)(void*);

    void (*mostrar)(void*);

} ICola;

// ===== ABSTRACTA =====

typedef struct {

    ICola* ops;

} Base;

// ===== NODO =====

typedef struct Nodo{

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo *f, *r; // frente y final

} Cola;
```



Código 7 .

Código Erróneo

```
#include <stdio.h>
#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {
    void (*enqueue)(void*, int)
    int (*dequeue)(void*);
    void (*mostrar)(void*);
} ICola;

// ===== ABSTRACTA =====

typedef struct {
    ICola ops; // ERROR: debería ser puntero
} Base;

// ===== NODO =====

typedef struct Nodo{
    int dato
    struct Nodo sig; // ERROR: debería ser puntero
} Nodo;

// ===== CLASE =====

typedef struct {
    Base base;
```

Código 7: Este programa muestra cómo usar una cola en un caso real:

- Permite gestionar datos de forma dinámica mediante nodos enlazados
    - Aplica operaciones fundamentales como enqueue y dequeue
    - Integra estructuras de datos con un enfoque orientado a objetos
- Es eficiente y funcional

## Código 7 BIS.

## Código Correcto

```
#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct{

    void (*ingresar)(void*);

    void (*insertarInicio)(void*, int);

    void (*mostrar)(void*);

} ILista;

// ===== ABSTRACTA =====

typedef struct{

    ILista* ops;

} Base;

// ===== NODO =====

typedef struct Nodo{

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct{

    Base base;

    Nodo* head;

} Lista;
```

## Código 7 BIS.

## Código Erróneo

```

#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct{

    void (*ingresar)(void*)

    void (*insertarInicio)(void*, int);

    void (*mostrar)(void*);

} ILista;

// ===== ABSTRACTA =====

typedef struct{

    ILista ops; // ERROR: debería ser puntero

} Base;

// ===== NODO =====

typedef struct Nodo{

    int dato

    struct Nodo sig; // ERROR: debería ser puntero

} Nodo;

// ===== CLASE =====

typedef struct{

    Base base;

```

Código 7 BIS: Este programa muestra cómo usar una lista en un caso real:

- Permite registrar y gestionar elementos de una colección
- Aplica operaciones básicas como inserción y visualización de datos
- Integra estructuras de datos con un enfoque modular

Es sencillo y funcional

## Código 8.

## Código Correcto

```
#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*push)(void*, int);

    int (*pop)(void*);

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} IPila;

// ===== ABSTRACTA =====

typedef struct {

    IPila* ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo* top;

} Pila;
```

## Código 8.

```

Código Erróneo

#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*push)(void*, int)

    int (*pop)(void*);

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} IPila;

// ===== ABSTRACTA =====

typedef struct {

    IPila ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato

    struct Nodo sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

```

Código 8: Este programa muestra cómo usar una pila en un caso real:

- Permite ingresar y almacenar datos dinámicamente mediante nodos enlazados
    - Aplica operaciones fundamentales como push y pop
  - Integra estructuras de datos con un enfoque orientado a objetos
- Es dinámico y eficiente

## Código 9

Código Correcto

```
#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*enqueue)(void*, int);

    int (*dequeue)(void*);

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} ICola;

// ===== ABSTRACTA =====

typedef struct {

    ICola* ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo *front, *rear;

} Cola;
```

## Código 9.

## Código Erróneo

```

#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*enqueue)(void*, int)

    int (*dequeue)(void*);

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} ICola;

// ===== ABSTRACTA =====

typedef struct {

    ICola ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato

    struct Nodo sig;

} Nodo;

// ===== CLASE =====

typedef struct {

```

Código 9: Este programa muestra cómo usar una cola en un caso práctico:

- Permite simular la atención de elementos en orden de llegada (FIFO)
  - Gestiona datos dinámicamente mediante memoria enlazada
  - Aplica principios de estructuras de datos y modularidad

Es claro, útil y fácil de implementar

## Código 10.

Código Correcto

```
#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*insertar)(void*, int);

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} ILista;

// ===== ABSTRACTA =====

typedef struct {

    ILista* ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato;

    struct Nodo* sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo* head;

} Lista;
```



## Código 10

```

Código Erróneo

#include <stdio.h>

#include <stdlib.h>

// ===== INTERFAZ =====

typedef struct {

    void (*insertar)(void*, int)

    void (*mostrar)(void*);

    void (*ingresar)(void*);

} ILista;

// ===== ABSTRACTA =====

typedef struct {

    ILista ops;

} Base;

// ===== NODO =====

typedef struct Nodo {

    int dato

    struct Nodo sig;

} Nodo;

// ===== CLASE =====

typedef struct {

    Base base;

    Nodo head;

```

Código 10: Este programa muestra cómo usar una lista enlazada en un caso práctico:

- Permite capturar e insertar elementos de forma dinámica en memoria
  - Aplica operaciones básicas como inserción y recorrido de la lista
    - Utiliza estructuras enlazadas junto con un diseño modular

Es simple, flexible y eficiente